

Engenharia de Software II

Trabalho projeto Fênix

Arthur Pontes Labre Feitosa

Bruno Eduardo Urbano Mota

Gabriel Aquino Carvalho Rodrigues

Ingrid Costa Sousa

João Vítor Piagem Pereira

Luís Eduardo dos Santos Gonçalves

Pedro Roberto Ribeiro Bandeira Labre

1. Comprometimento com Prazo Irrealista

Problema: O gerente de projetos prometeu ao cliente a entrega de um sistema complexo em um prazo de apenas quatro meses, sem nenhuma consulta prévia à equipe técnica para avaliar a viabilidade.

Impacto: Esta decisão criou uma pressão artificial que impossibilitou um planejamento adequado, forçando a equipe a tomar decisões técnicas apressadas e de curto prazo e estabelecendo um cronograma irreal desde o início do projeto.

Solução: Conduzir sempre uma análise de viabilidade técnica, envolvendo a equipe de desenvolvimento, para estimar esforço, riscos e complexidade antes de definir e se comprometer com qualquer prazo com o cliente.

2. Mudanças Constantes de Escopo sem Controle

Problema: Durante o projeto, novas funcionalidades significativas, como um módulo de gestão de fornecedores e integração com um sistema legado, foram solicitadas pelo cliente e aceitas sem qualquer replanejamento de cronograma ou reavaliação dos recursos necessários.

Impacto: O escopo aumentou de forma descontrolada, tornando o trabalho caótico, sobrecarregando a equipe e comprometendo a qualidade do produto que estava sendo desenvolvido.

Solução: Implementar um processo formal de controle de mudanças. Cada nova solicitação deve ser analisada quanto ao seu impacto em prazo, custo e recursos, e então negociada com o cliente antes de ser incorporada ao escopo do projeto.

3. Falta de Levantamento e Documentação de Requisitos

Problema: Não houve uma fase formal de levantamento de requisitos. A única orientação para a equipe foi um e-mail vago do gerente com anotações de uma reunião.

Impacto: O desenvolvimento foi baseado em interpretações subjetivas , o que gerou retrabalho constante e um produto final que não atendia às expectativas reais do cliente.

Solução: Implementar uma fase dedicada ao levantamento e análise de requisitos, utilizando técnicas como entrevistas e workshops. Documentar os requisitos de forma clara e estruturada, garantindo que sejam validados pelo cliente antes do início do desenvolvimento.

4. Desenvolvimento de Interface sem Protótipo ou Design

Problema: A interface do usuário foi desenvolvida de forma improvisada, baseando-se apenas na opinião pessoal do desenvolvedor júnior, sem a criação de protótipos para validação com o cliente.

Impacto: A abordagem resultou em retrabalho frequente e custoso , além de problemas graves de usabilidade, como uma interface com cores ilegíveis (verde-limão) que foi rejeitada pelos usuários finais.

Solução: Criar protótipos de baixa e alta fidelidade da interface e validá-los com o cliente e usuários finais antes de iniciar o desenvolvimento. Elaborar um guia de estilo para garantir a consistência visual e funcional.

5. "Hardcoding" de Dados Críticos no Código-Fonte

Problema: Informações e regras de negócio importantes, como taxas de impostos e nomes de filiais, foram fixadas diretamente no código-fonte.

Impacto: Esta prática tornou o sistema inflexível e de difícil manutenção , pois qualquer alteração nesses valores exigiria uma intervenção direta no código e uma nova implantação do sistema.

Solução: Externalizar todas as configurações e parâmetros de negócio do código-fonte. Utilizar arquivos de configuração, variáveis de ambiente ou um banco de dados para armazenar esses valores, idealmente com uma interface administrativa para que usuários autorizados possam gerenciá-los.

6. Ausência Total de Testes Unitários e Automatizados

Problema: A equipe não escreveu nenhum teste unitário para o código, justificando a decisão com a falta de tempo e a pressão do prazo.

Impacto: Falhas simples não foram detectadas durante o desenvolvimento e só apareceram na fase final ou em produção, aumentando o custo de correção e comprometendo a confiabilidade do sistema.

Solução: Integrar a prática de testes unitários no processo de desenvolvimento. Adotar práticas como o Desenvolvimento Guiado por Testes (TDD) e implementar um pipeline de integração contínua (CI) que execute os testes automaticamente a cada nova alteração no código.

7. Envolvimento Tardio e Inadequado da Equipe de QA

Problema: A profissional de Qualidade (QA) foi acionada apenas uma semana antes do prazo final de entrega. Ela teve que validar o sistema sem um plano de testes ou uma documentação de requisitos como base.

Impacto: O teste tornou-se uma atividade apressada, superficial e ineficiente. Dezenas de bugs críticos foram descobertos tarde demais para serem corrigidos adequadamente antes do lançamento.

Solução: Integrar a equipe de QA desde o início do projeto, para que participe da definição dos requisitos e elabore o plano de testes. Adotar uma abordagem de teste contínuo, realizando validações em paralelo com o desenvolvimento.

8. Controle de Versão Caótico e Inadequado

Problema: Não havia um fluxo de versionamento definido. Um desenvolvedor fez um commit diretamente na branch principal, sobrescrevendo e apagando o trabalho de uma semana da desenvolvedora sênior.

Impacto: A falta de um processo adequado resultou na perda de código, que levou dois dias para ser recuperado, causando retrabalho massivo e atrasos no cronograma.

Solução: Adotar um fluxo de trabalho de versionamento estruturado, como o GitFlow. Definir uma política clara que exija que todo o desenvolvimento seja feito em branches separadas e que todo o código seja revisado antes de ser integrado à branch principal.

9. Ausência de Metodologias Ágeis e Comunicação Eficaz

Problema: O projeto avançou sem o uso de qualquer abordagem ágil, como Scrum ou Kanban. Não havia cerimônias como reuniões diárias, planejamento de sprints ou reuniões de acompanhamento para garantir a transparência e o alinhamento.

Impacto: O projeto avançou sem visibilidade sobre o progresso, riscos ou impedimentos. A comunicação desorganizada impediu ajustes a tempo e causou descoordenação entre as equipes de front-end e back-end.

Solução: Adotar uma metodologia ágil e implementar cerimônias essenciais, como reuniões diárias para sincronização, planejamento de sprints para definir metas e retrospectivas para melhoria contínua.

10. Problemas Graves de Performance e Integridade de Dados

Problema: O sistema apresentava falhas técnicas críticas. Relatórios em "tempo real" demoravam cinco minutos para carregar e o sistema travava e corrompia os dados do estoque ao tentar processar 100 itens simultaneamente.

Impacto: Essas falhas tornaram o sistema lento e inviável para o uso operacional diário, causando a perda de dados críticos para o negócio e frustração generalizada nos usuários.

Solução: Incorporar testes de performance e de carga desde as fases iniciais do desenvolvimento. Realizar otimizações de banco de dados e de código focadas nos requisitos de desempenho e garantir que as transações de dados sejam robustas para evitar corrupção.

11. Processo de Implantação Manual e Arriscado

Problema: A instalação do software foi realizada manualmente pela desenvolvedora sênior, de madrugada, diretamente no servidor do cliente, sem automação, ambiente de homologação ou um plano de reversão.

Impacto: O processo elevou o risco de falha humana e dificultou a reversão em caso de problemas. Problemas de compatibilidade e desempenho que poderiam ter sido detectados em um ambiente de teste só surgiram na produção.

Solução: Implementar um pipeline de Integração e Entrega Contínua (CI/CD) para automatizar os processos de teste e deploy. Utilizar ambientes de homologação idênticos ao de produção para testes finais.

12. Equipe Subdimensionada e Dependência Excessiva de um Desenvolvedor

Problema: Apenas três pessoas foram designadas para um sistema corporativo de grande porte. A desenvolvedora sênior, Ana, acumulava as funções de arquiteta, programadora principal e responsável pela implantação.

Impacto: Essa estrutura criou um gargalo no desenvolvimento e um ponto único de falha. A sobrecarga levou à exaustão e ao pedido de demissão da desenvolvedora, representando uma perda de conhecimento crítico para a empresa.

Solução: Dimensionar a equipe de acordo com a complexidade do projeto. Promover a partilha de conhecimento através de práticas como programação em par e revisões de código, e documentar a arquitetura para evitar que o conhecimento fique concentrado em uma única pessoa.

13. Cultura de Medo e Falta de Orientação

Problema: O desenvolvedor júnior não recebeu mentoria adequada e, por falta de segurança psicológica, não se sentia à vontade para reportar bugs que encontrava no código da colega sênior, por receio de interrompê-la.

Impacto: Problemas identificados não foram comunicados e se acumularam silenciosamente até chegar à produção. A falta de um ambiente seguro e de reconhecimento também desmotivou o desenvolvedor júnior.

Solução: Fomentar uma cultura de segurança psicológica onde a equipe se sinta segura para apontar erros. Implementar a revisão de código como uma prática padrão para que o feedback seja construtivo e parte do processo. Designar mentores para apoiar os profissionais em início de carreira.

14. Falha Operacional e Prejuízo Financeiro

Problema: O acúmulo de falhas técnicas e de gestão resultou em um produto final que, em produção, se mostrou lento, instável e impedia os funcionários de realizar operações básicas de venda e estoque. Um bug crítico permitiu a venda de produtos que não existiam em estoque.

Impacto: O cliente sofreu prejuízo financeiro direto e teve suas operações paralisadas. Isso destruiu a confiança no sistema, levou ao cancelamento do contrato, a uma ameaça de processo judicial e abalou severamente a reputação da empresa no mercado.

Solução: A solução para evitar este resultado catastrófico é a aplicação rigorosa e combinada de todas as soluções anteriores: um ciclo de vida de desenvolvimento de software maduro que previna a ocorrência dos problemas técnicos e gerenciais listados.